

советы по практике

я собрала тут все заметки из конспектов по практике, у Лени разбито на вопросы, у меня просто скопом можно посмотреть

создание таблицы

```
CREATE TABLE Students
(
    ID_student SERIAL, (SERIAL это диалект)
    Course INTEGER,
    Group INTEGER,
    ...
);
```

char - последовательность символов, вводим сколько их будет(size). Если вводим меньше, остальное заполняется пробелами.

varchar - послед-ть символов, сколько вводим, столько и хранится. (динамически тип)

ограничения:

1. NOT NULL - всегда на уровне столбца
2. UNIQUE
3. PK
4. FK
5. CHECK

+DEFAULT (не ограничение, но схоже по написанию) - на уровне столбца, т.к. **всегда** на 1 атрибут

огр-ия на уровне таблицы:

```
ALTER TABLE Students
ADD CONSTRAINT [PK_Students_ID_student] PRIMARY KEY (ID_student)
```

```
ALTER TABLE Students
ADD CONSTRAINT FOREIGN KEY(ID_fac) REFERENCES Faculty(ID_faculty)
```

- задается всегда после создания обеих таблиц

```
ALTER TABLE Students
ADD (ограничение / атрибут)
```

```
ALTER TABLE Students
DROP (ограничение / атрибут)
```

```
ALTER TABLE Students
MODIFY (ограничение )
```

SELECT

```
SELECT *
[FROM
WHERE
...]
```

Что мб во фразе WHERE:

1. IN / NOT IN - если будет NULL значение, все упадет, лучше EXISTS
2. LIKE / NOT LIKE
3. BETWEEN .. AND .. / NOT BETWEEN .. AND ..
4. IS NULL / NOT IS NULL
5. логические операторы <, >, =, .., NOT, AND, OR
6. подзапрос (всегда в скобках)
7. CASE

Что мб после SELECT:

1. константа
SELECT 'отличник', ...
2. конкатенация строк
SELECT Surname + ' ' + Name + ' ' + ...

SELECT **CONCAT**(Surname, ' ', Name, ' ', **SUBSTRING**(Patronymic 1, 1))

3. логич/арифметич условие

SELECT Corse * 6

4. агрегирующая функция

5. подзапрос (всегда в скобках)

DISTINCT применяется КО ВСЕЙ строке, т.е. если (DISTINCT Course, Group - будут уникальные пары). Это НЕ ФУНКЦИЯ, скобочки после нее не нужны.

% - означает любое количество символов, в т.ч. 0

___ - означает любой единичный символ, в т.ч. и пробел

Чтобы % и ___ были символами, используем

э к р а н и р о в а н и е - действует на следующий за ним 1 символ

WHERE email LIKE '%f#%mail.com' **ESCAPE '#'**

INSERT

1 вариант	2 вариант
INSERT INTO <i>имятабл</i> [<i>имястолбца</i>] VALUES (список значений); если без списка атрибутов, то вставку пишем в том порядке, в котором атрибуты в таблице идут	INSERT INTO <i>имятабл</i> [<i>имястолбца</i>] (SELECT ...); т.е. без VALUES
INSERT INTO <i>имятабл</i> [<i>имястолбца</i>] VALUES(1, 3, (SELECT ..)) - следим, чтобы подзапрос вернул 1 результат!	

UPDATE, DELETE

```
UPDATE Students
```

```
SET имя_столбца = значение, имя_столбца = значение..
```

```
[WHERE ..]
```

```
DELETE
FROM Students
```

TRUNCATE [TABLE] Students --применяется только для таблиц

DELETE	TRUNCATE
построчно удаляет все данные	перемещает указатель заполненности таблицы на начало таблицы
реагирует на триггеры (а они могут быть навешаны на каждую строчку)	не обрабатывает триггеры
работает медленнее	работает быстрее

Сортировка ORDER BY

```
SELECT
FROM
WHERE
ORDER BY - по умолчанию по возрастанию и в лексикографическом порядке
```

ASC, DESC -явное указание для сортировки по возрастанию и убыванию соответственно.

Можно:

1. номера столбцов
ORDER BY 1, 2, 3 - номера столбцов в SELECT
2. алиасы
ORDER BY курс

Преобразование типов

CAST (имя_столбца AS тип) - стандарт

TO_DATE(..)
TO_NUMBER(..)
TO_CHAR(..)
стипендия * 1.0

EXTRACT (YEAR/MONTH/DAY FROM Date)

SUBSTRING(Name 1,1) - выделит одну букву из подстроки (начиная с, количество)

CASE

- как только произойдет полное совпадение, вернет результат THEN
- результаты должны быть одного типа

1 вариант

```
CASE атрибут
    WHEN знач1 THEN ...
    WHEN знач2 THEN ..
    [ELSE ...]
END
```

2 вариант

```
CASE
    WHEN лог_услов1 (мб подзапрос) THEN ...
    [ELSE ...]
END
```

COALESCE

COALESCE(значение, 'нет') - может принимать несколько параметров, остановится на первом не NULL. Вернуть должен тот тип, который у параметров.

Групповые функции

- агрегирующая функция применяется к группе строк, возвращает одно значение

- все функции, кроме COUNT() игнорируют значения NULL. COUNT(s.*) посчитает даже тех, у кого NULL значения.

1. MAX()
2. MIN()
3. SUM()
4. AVG()
5. COUNT()

Т.е.

SUM(стипендия)/COUNT(студенты) - посчитает среднее значение в т.ч. для тех, у кого нет стипенды

AVG(стипендия) - посчитает только для тех, у кого есть стипендия

AVG() наследует тип, к которому применяется

AVG(Стипендия * 1.0) - неявное приведение типов, можно и явным.

JOIN

- сначала указываем таблицы, присоединение которых отсечет максимальное кол-во данных. СУБД соединяет по 2 таблицы: соединила, подключает полученную с следующей.

A JOIN B ON ... - присоединит

A LEFT JOIN B ON ...

A RIGHT JOIN B ON ...

A FULL JOIN B ON ... - войдут все значения A, все значения B и если есть пересечения, то они тоже, без повторений.

A CROSS JOIN B ON - комбинации все со всеми.

GROUP BY

- если функция агрегирования и JOIN
- пишем все параметры из SELECT, кроме тех, на которые функции агрегирования. + мб атрибуты, которых нет в SELECT

Подзапросы

всегда пишутся в ()

- подзапрос может тянуть данные из внешнего запроса - это **коррелирующий** подзапрос

SELECT ()
FROM () JOIN ()
WHERE ()
GROUP BY ТУТ НЕЛЬЗЯ
HAVING ()
ORDER BY ТУТ НЕЛЬЗЯ

EXISTS / NOT EXISTS

- **ТОЛЬКО** с коррелирующими подзапросами в фразе WHERE
- стабилен для NULL значений, в отличие от IN - там все упадет
- часто забывают писать корреляцию!
- хорошо работает на **большом** объеме данных
- когда надо проверить, что чего-то нет (не сдавших предмет, ..)
- если в подзапросе EXISTS вылезла группировка, значит что-то не так, надо по-другому идти

```
SELECT s.*  
FROM Students s  
WHERE NOT EXISTS( SELECT 1  
                  FROM Успеваемость y  
                  WHERE y.ID_stud = s.ID_stud)
```

Операции над множествами

UNION / UNION ALL - объединение результатов двух запросов. с неявной сортировкой.

UNION	UNION ALL
без дубликатов	оставляет дубликаты, работает быстрее

- столбцы в результирующей таблице именуются по первому selecty
- запросы, к которым применяется операции над множествами, должны быть **совместимы** (кол-во столбцов, попарно совпадают типы)
- данные в итоговой таблице будут вперемешку. можно ORDER BY по названию столбцов (при UNION нельзя), алиасу (при UNION нельзя), номеру.

INTERSECT - пересечение с неявной сортировкой

EXCEPT / MINUS - и то и то стандарт с неявной сортировкой.

ALL / ANY (SOME) - ANY = SOME!!

- после них всегда подзапрос

ALL	ANY
= ALL -ист, если подзапрос выбрал 1 знач, либо выбрал много, но они все равны (одинаковые)	= ANY оно же IN
<> ALL	<> ANY оно же "не равно хотя бы одному"
> ALL оно же < MIN	> ANY оно же >MIN (хоть одного)
>= ALL оно же >= MAX	< ANY оно же < MAX
<= ALL оно же <=MIN	>= ANY оно же >= MIN
	<= ANY оно же <= MAX